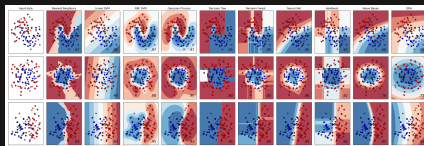


Máquinas de vectores de soporte y selección de modelos

Aprendizaje Automatico



Marco Teran
EAFIT

2025

Contenido

- 1 Objetivos de aprendizaje
- 2 K-Nearest Neighbors (KNN)
 - Complejidad y sobreajuste
- 3 Support Vector Machines (SVM)
 - SVM Lineal
 - Kernel Trick
 - Selección de modelos
 - SVM para regresión
- 4 Conclusiones

Objetivos de aprendizaje

Objetivos de aprendizaje

- **Dominar clasificación no lineal:** KNN y SVM para problemas complejos
- **Comprender el dilema bias-variance:** sobreajuste vs subajuste
- **Implementar validación cruzada:** *k-fold* y estratificación
- **Optimizar hiperparámetros:** *Grid Search* y *Random Search*
- **Aplicar kernel trick:** de lineal a no lineal sin transformación explícita
- **Diagnosticar complejidad del modelo:** curvas de aprendizaje

Filosofía central

Balancear simplicidad interpretable con capacidad de capturar patrones no lineales

K-Nearest Neighbors: Clasificación basada en vecinos

¿Qué es KNN?

Aprendizaje basado en instancias

- No construye un modelo explícito durante entrenamiento
- **Lazy learning**: almacena todos los ejemplos
- Clasifica nuevas instancias por **votación de mayoría**
- Simple pero sorprendentemente efectivo

Intuición

Dime quiénes son tus vecinos y te diré quién eres

Fundamentos de KNN

Algoritmo de clasificación:

- 1 Calcular distancia a todos los puntos de entrenamiento
- 2 Seleccionar los k vecinos más cercanos
- 3 Votación por mayoría
- 4 Asignar clase más frecuente

Parámetro clave: k : número de vecinos

- k pequeño $\rightarrow$ alta complejidad
- k grande $\rightarrow$ baja complejidad

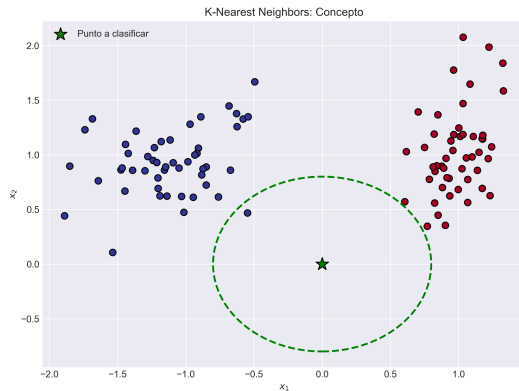


Figura 1: Ilustración del algoritmo KNN

Métricas de distancia

Distancia Euclidiana (más común):

$$d(x_i, x_j) = \sqrt{\sum_{l=1}^n (x_{i,l} - x_{j,l})^2}$$

Distancia Manhattan:

$$d(x_i, x_j) = \sum_{l=1}^n |x_{i,l} - x_{j,l}|$$

Distancia Minkowski (generalización):

$$d(x_i, x_j) = \left(\sum_{l=1}^n |x_{i,l} - x_{j,l}|^p \right)^{1/p}$$

Importante

Escalar features es crítico - diferentes unidades afectan distancias

Efecto del hiperparámetro k

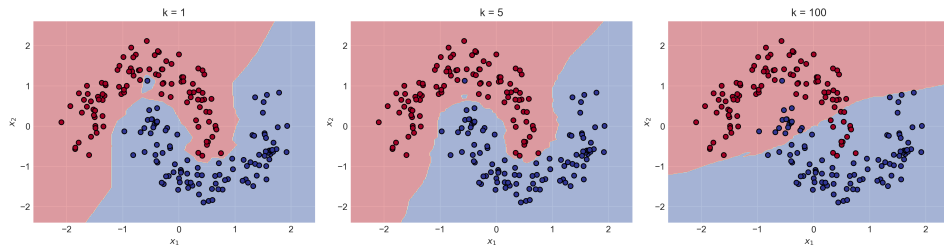


Figura 2: Frontera de decisión para diferentes valores de k

- $k = 1$: memoriza datos, máximo sobreajuste
- $k = 5$: balance típico
- $k = 100$: suaviza demasiado, subajuste

El dilema de la complejidad

Sobreajuste vs Subajuste

Sobreajuste (Overfitting):

- Modelo muy complejo
- Aprende ruido
- Buen error en train
- Mal error en test
- k pequeño en KNN

Subajuste (Underfitting):

- Modelo muy simple
- No captura patrones
- Mal error en train
- Mal error en test
- k grande en KNN

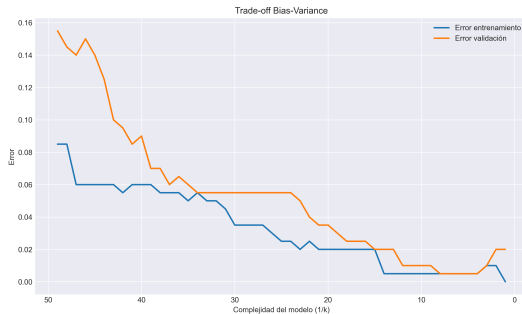


Figura 3: Trade-off bias-variance

Curvas de aprendizaje

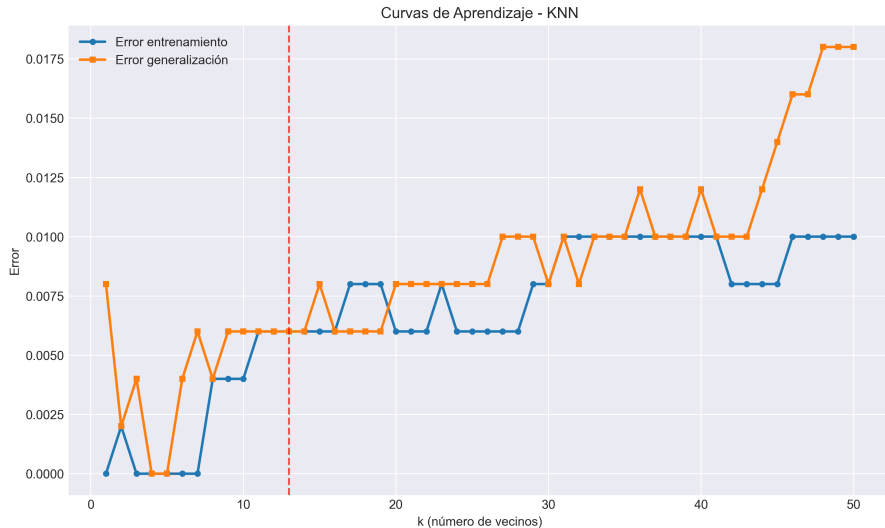


Figura 4: Error de entrenamiento vs generalización

Ventajas y desventajas de KNN

Ventajas:

- Simple de entender e implementar
- Sin fase de entrenamiento
- Naturalmente multiclase
- Efectivo para datasets pequeños
- No asume distribución de datos

Desventajas:

- Predicción lenta ($O(nm)$)
- Requiere mucha memoria
- Sensible a features irrelevantes
- Sufre con alta dimensionalidad
- Necesita escalado de features

Maldición de la dimensionalidad

En altas dimensiones, todos los puntos están *lejos* entre sí

Support Vector Machines: Máximo margen

¿Qué es SVM?

Máquina de Vectores de Soporte

- Algoritmo de clasificación supervisada
- Busca el **hiperplano óptimo** de separación
 - La mejor línea o límite de decisión que ayuda a clasificar los puntos de datos en el espacio n-dimensional
 - La dimensión del hiperplano depende de las características presentes en el conjunto de datos.
- Maximiza el **margen** entre clases
- Robusto y versátil

Filosofía

No solo separar clases, sino maximizar la distancia de seguridad

Contexto histórico

- **1963:** Vapnik y Lerner - Primer SVM lineal
- **1992:** Boser, Guyon, Vapnik - Kernel trick
- **1995:** Cortes y Vapnik - Soft margin
- **1990s-2000s:** Dominan Machine Learning
- **Hoy:** Herramienta fundamental para problemas complejos

¿Por qué siguen relevantes?

Garantías teóricas sólidas, interpretabilidad, eficiencia en dimensiones medias

Intuición geométrica

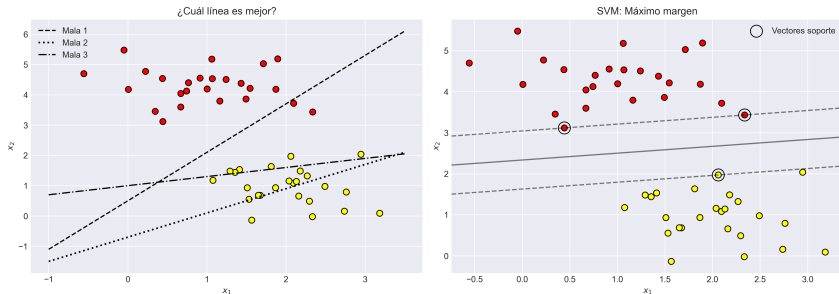


Figura 5: El problema de máximo margen

Pregunta clave: ¿Cuál línea es mejor?

- Todas separan las clases
- SVM elige la que maximiza distancia mínima
- Más robusta a outliers y nueva data

SVM Lineal: Clasificación de margen grande

Hard Margin SVM

Formulación matemática:

El hiperplano de decisión:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

Objetivo: Maximizar margen $= \frac{2}{\|\mathbf{w}\|}$

Equivale a minimizar:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

Sujeto a:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i = 1, \dots, m$$

Limitación

Solo funciona si datos son linealmente separables

Vectores de soporte

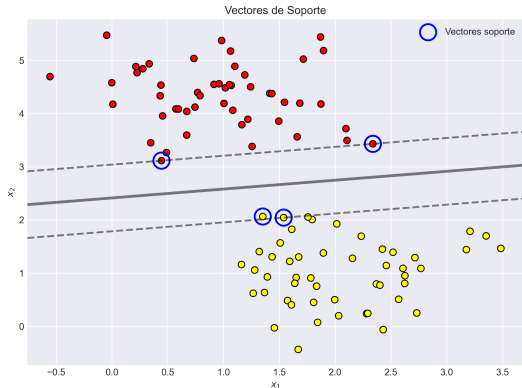
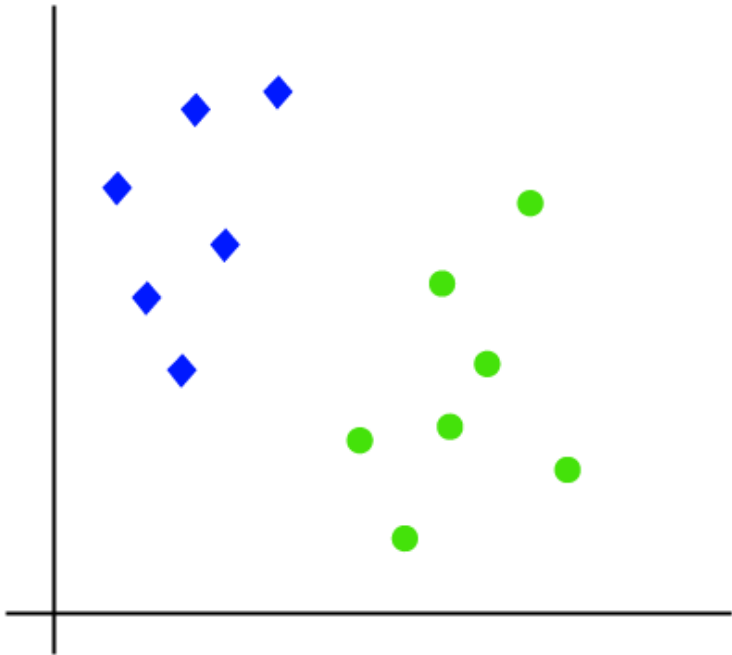
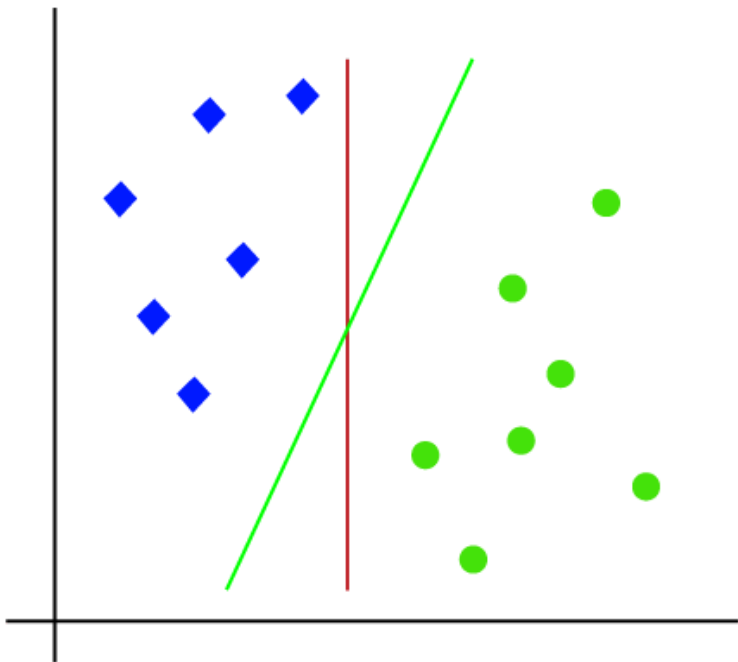
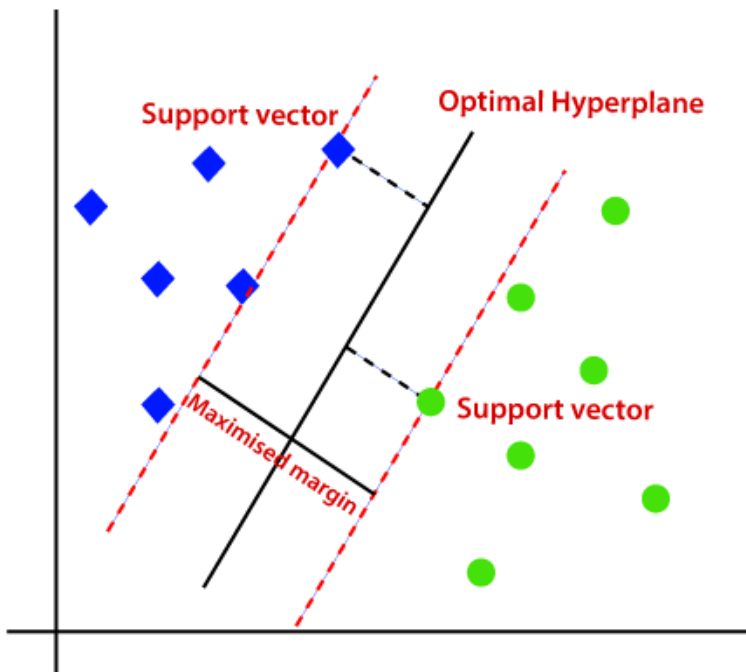


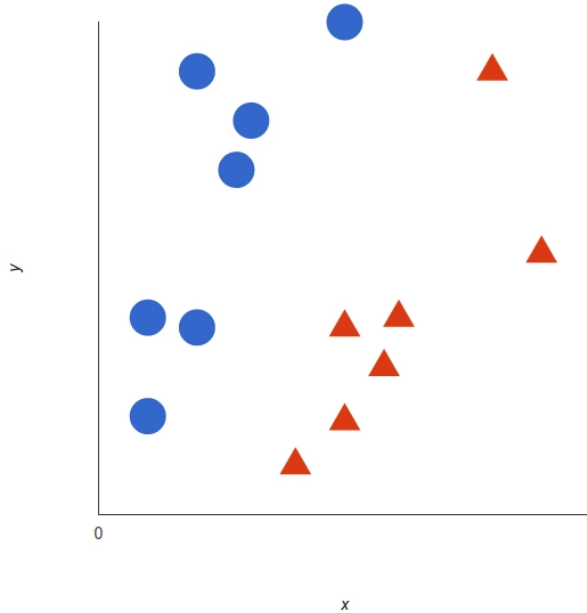
Figura 6: Vectores de soporte (circled points)

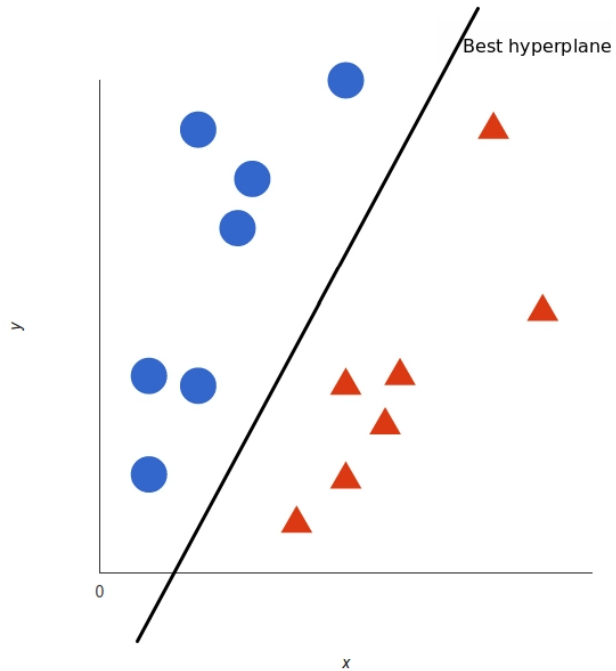
- Son los puntos o vectores de datos más cercanos al hiperplano y que afectan la posición del mismo
- Estos vectores soportan al **hiperplano**, de ahí su nombre **Vectores de Soporte**
- **Propiedad clave:** Solo los puntos en el margen determinan el hiperplano óptimo

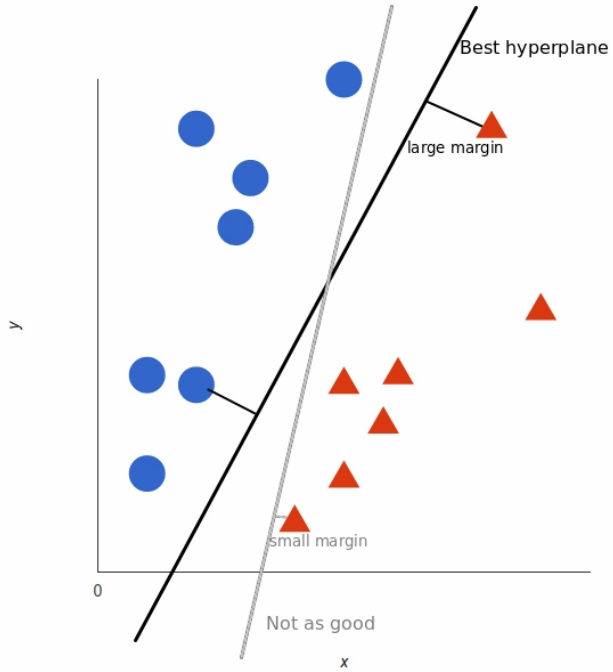












Soft Margin SVM

Problema: Datos raramente son perfectamente separables

Solución: Permitir violaciones controladas

$$\min_{\mathbf{w}, b, \vec{\xi}} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$

Sujeto a:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \quad \text{y} \quad \xi_i \geq 0$$

- ξ_i : **slack variables** (variables de holgura)
- C : **parámetro de regularización**
 - C grande $\rightarrow$ pocos errores, margen pequeño
 - C pequeño $\rightarrow$ más errores, margen grande

Efecto del parámetro C

Efecto del parámetro C en SVM

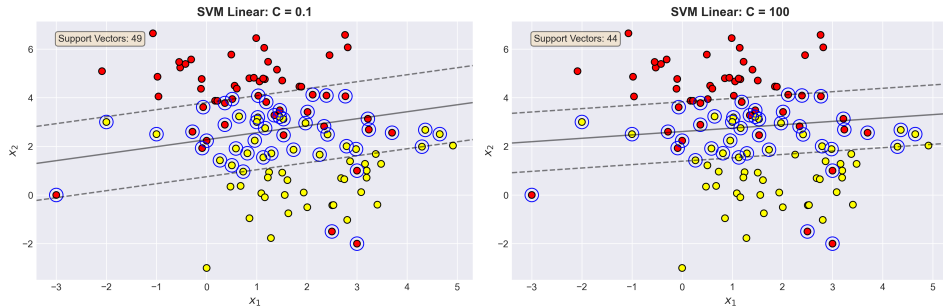


Figura 7: $C = 0.1$ (izq) vs $C = 100$ (der)

Trade-off

C controla el balance entre margen grande y pocas violaciones

Sensibilidad al escalado

Efecto del Escalado en SVM: ¿Por qué StandardScaler es crucial?

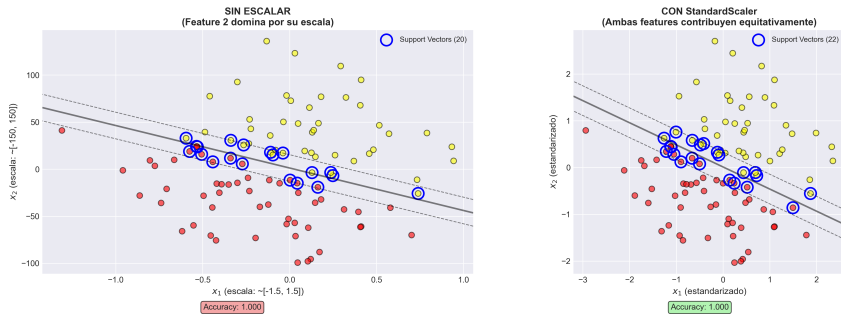
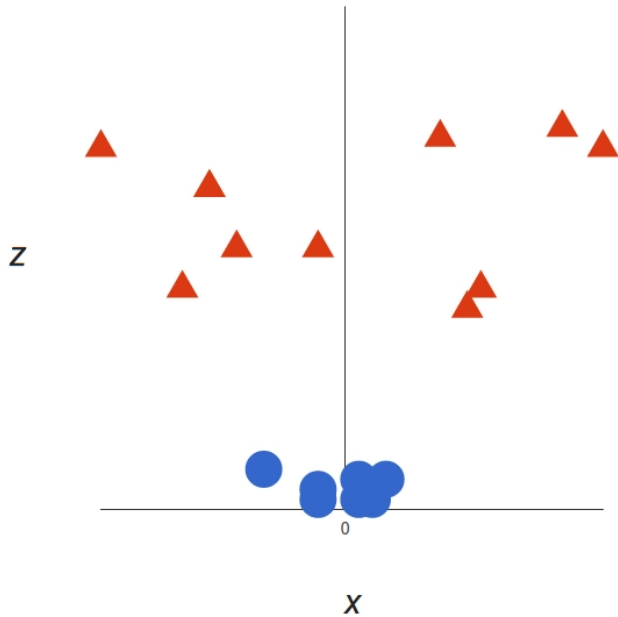


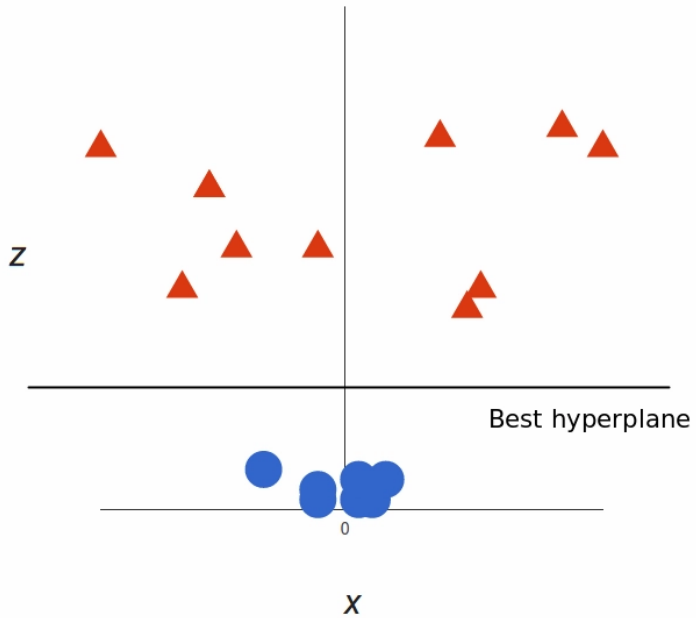
Figura 8: Sin escalar (izq) vs escalado (der)

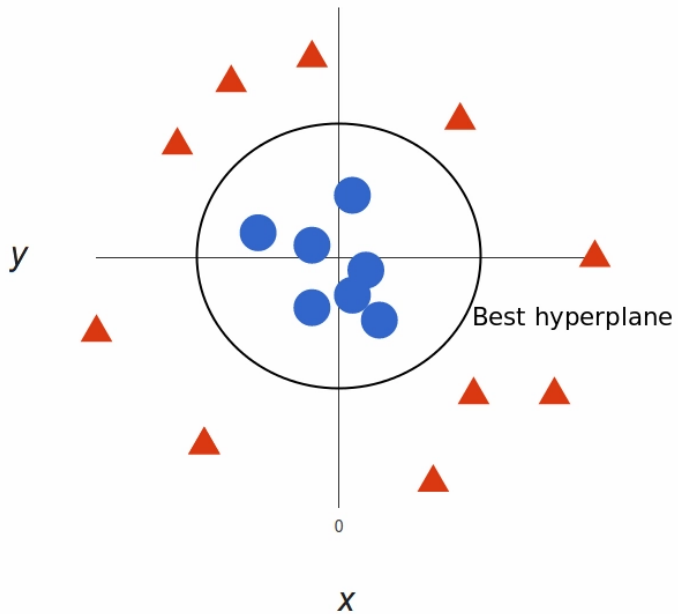
Regla de oro

SIEMPRE escalar features antes de entrenar SVM

De lineal a no lineal







El problema de la no linealidad

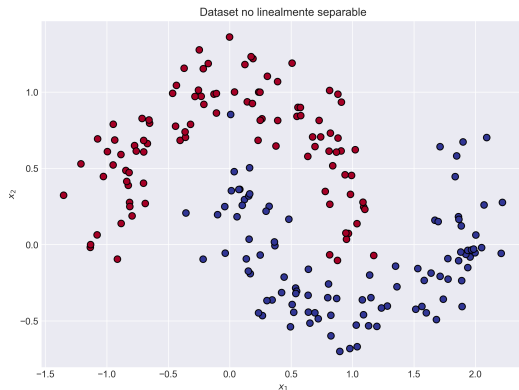


Figura 9: Dataset no linealmente separable

Pregunta: ¿Cómo separar estos datos?

■ SVM lineal falla

■ ¿Agregar features polinómicas? ¡Explosión combinatoria!

Feature mapping

Idea: Transformar a espacio de mayor dimensión

$$\phi: \mathbb{R}^n \rightarrow \mathbb{R}^m \quad (m \gg n)$$

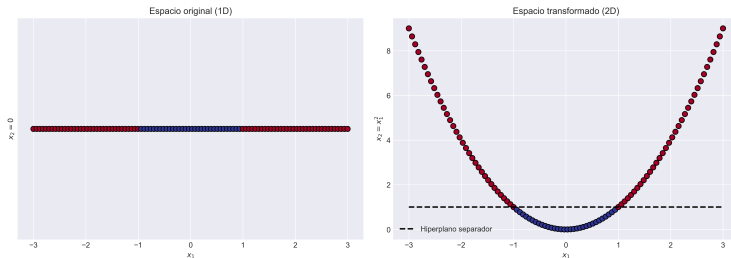


Figura 10: Mapeo de 1D a 2D

Problema

Calcular $\phi(x)$ explícitamente es costoso o imposible

El truco del kernel

Insight brillante:

No necesitamos calcular $\phi(\mathbf{x})$ explícitamente

Solo necesitamos productos punto: $\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$

Kernel function

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

Calcula producto punto en espacio transformado ¡sin transformar!

Ejemplo:

$$K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y})^2 = \phi(\mathbf{x})^T \phi(\mathbf{y})$$

donde ϕ mapea a espacio de dimensión $\binom{n+1}{2}$

Kernels comunes

Kernel Lineal:

$$K(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \mathbf{y}$$

Kernel Polinomial:

$$K(\mathbf{x}, \mathbf{y}) = (\gamma \mathbf{x}^T \mathbf{y} + r)^d$$

Kernel RBF (Gaussiano):

$$K(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \|\mathbf{x} - \mathbf{y}\|^2)$$

Kernel Sigmoide:

$$K(\mathbf{x}, \mathbf{y}) = \tanh(\gamma \mathbf{x}^T \mathbf{y} + r)$$

Kernel polinomial

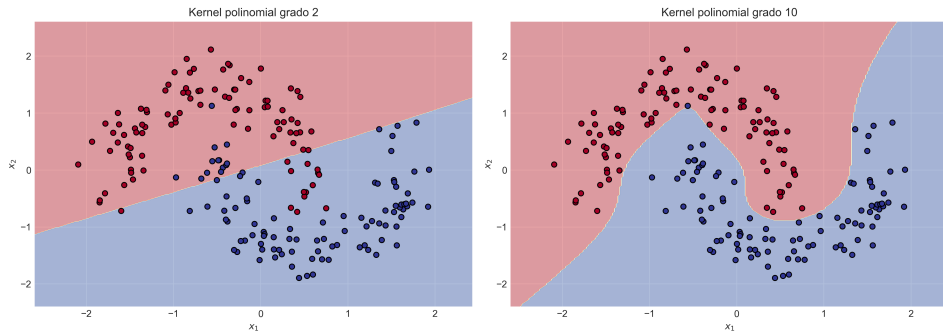


Figura 11: $d = 2$ (izq) vs $d = 10$ (der)

Hiperparámetros:

- d : grado del polinomio
- r : coef0 (término independiente)
- γ : escala del producto punto

Kernel RBF (Gaussiano)

$$K(x, y) = \exp(-\gamma \|x - y\|^2)$$

- **Más popular** para problemas no lineales
- Similar a funciones de base radial
- γ : controla influencia de cada muestra

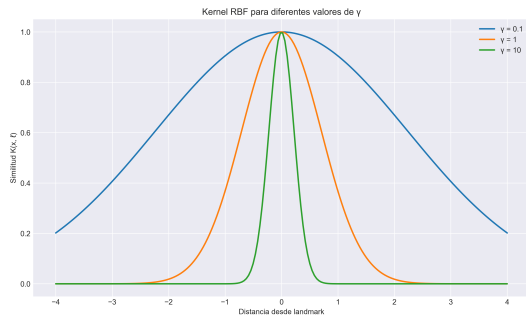


Figura 12: Función RBF para diferentes γ

Efecto de γ en RBF

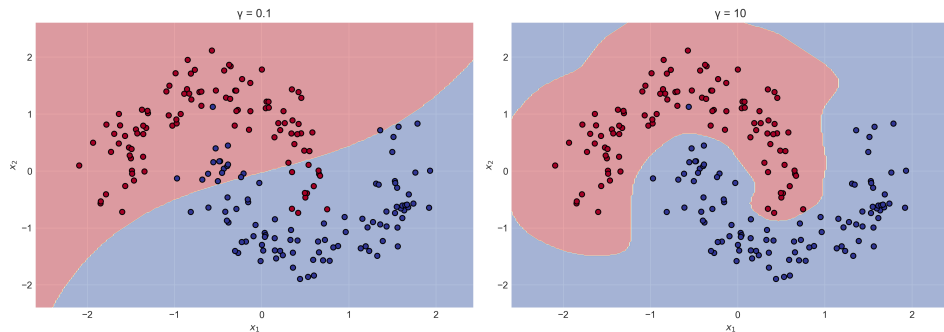


Figura 13: $\gamma = 0.1$ (izq) vs $\gamma = 10$ (der)

- γ pequeño $\rightarrow$ decisión suave (subajuste)
- γ grande $\rightarrow$ decisión compleja (sobreajuste)

Interacción C y γ

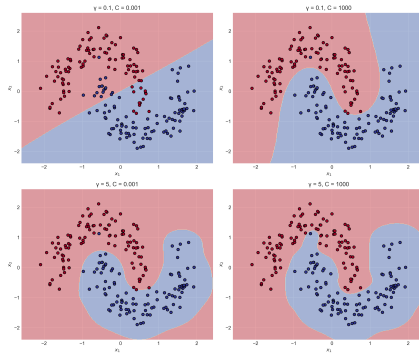


Figura 14: Efecto conjunto de C y γ

Optimización

Ambos parámetros deben ajustarse simultáneamente

Validación cruzada y Grid Search

El problema de la validación

Partición tradicional:

- Train: 70%
- Test: 30%

Problema: ¿Cómo ajustar hiperparámetros sin contaminar test?

Solución: Introducir conjunto de **validación**

- Train: 60%
- Validation: 20%
- Test: 20%

Flujo

Train → ajustar → Validation → evaluar → Test (una sola vez)

Validación cruzada K-Fold

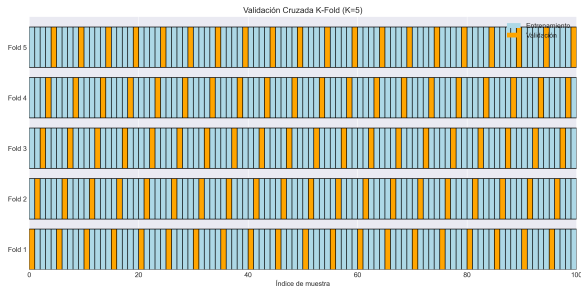


Figura 15: Validación cruzada de 5 pliegues

Algoritmo:

- 1 Dividir datos en k pliegues (típicamente 5 o 10)
- 2 Para cada pliegue:
 - Entrenar en $k - 1$ pliegues
 - Validar en pliegue restante
- 3 Promediar resultados

Validación estratificada

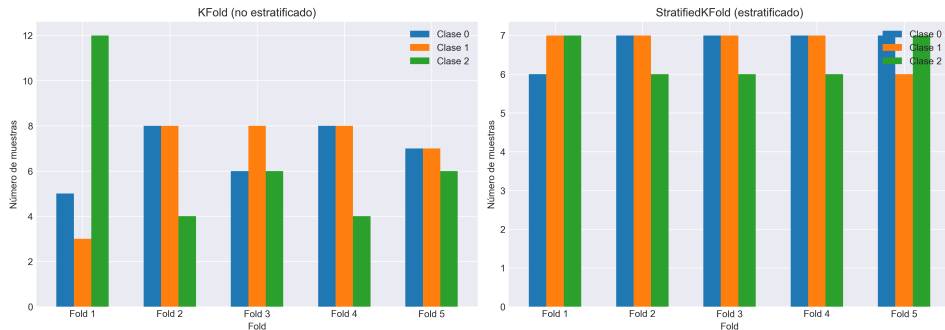


Figura 16: KFold vs StratifiedKFold

Estratificación

Cada pliegue mantiene la proporción de clases del dataset original

Grid Search

Búsqueda exhaustiva en malla de hiperparámetros

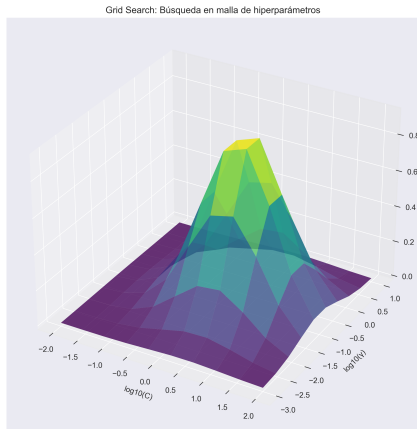


Figura 17: Grid Search 2D

Ejemplo SVM RBF:

- $C \in \{0.1, 1, 10, 100\}$
- $\gamma \in \{0.001, 0.01, 0.1, 1\}$
- Total: $4 \times 4 = 16$ combinaciones
- Con CV 5-fold: $16 \times 5 = 80$ entrenamientos

Visualización de Grid Search

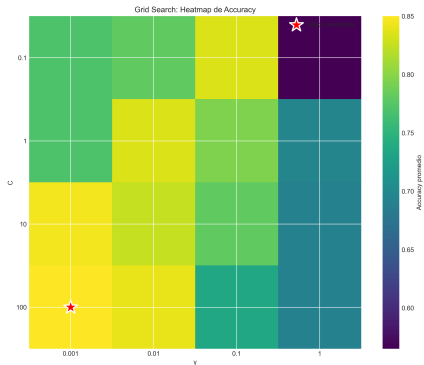


Figura 18: Heatmap de accuracy para diferentes (C, γ)

Interpretación:

- Colores claros → mejor accuracy
- Esquinas → regiones de sobre/subajuste
- Centro → zona de balance óptimo

Random Search

Alternativa eficiente a Grid Search

Grid Search:

- Exhaustivo
- Lento para muchos parámetros
- Puede perder óptimos

Random Search:

- Muestreo aleatorio
- Más eficiente
- Mejor cobertura

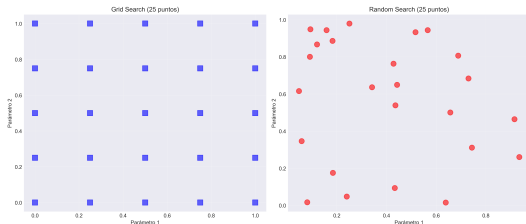


Figura 19: Grid vs Random Search

Support Vector Regression

SVM Regression

Objetivo invertido: Ajustar margen con máximo de puntos dentro

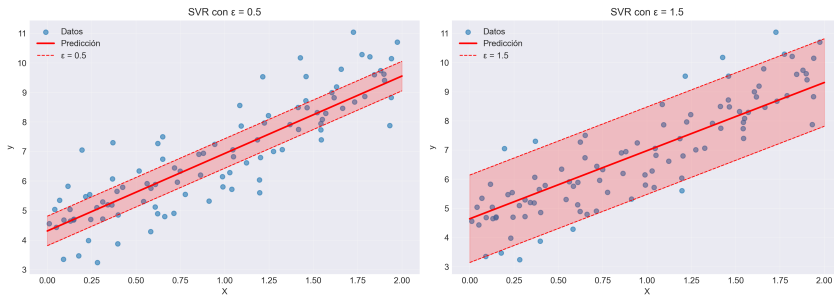


Figura 20: SVR con diferentes ϵ

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad |y_i - (w^T x_i + b)| \leq \epsilon$$

Aplicaciones de SVM

Clasificación:

- Reconocimiento facial
- Clasificación de texto
- Detección de spam
- Diagnóstico médico
- Bioinformática

Regresión:

- Predicción de series temporales
- Estimación de propiedades
- Control de calidad
- Pronóstico financiero

Ventaja competitiva

Excelente con datasets medianos y alta dimensión

Comparación: KNN vs SVM

Criterio	KNN	SVM
Fase entrenamiento	Ninguna	Costosa
Fase predicción	Lenta	Rápida
Memoria	Alta	Baja
Interpretabilidad	Alta	Media
No linealidad	Natural	Vía kernels
Escalabilidad	Mala	Buena (lineal)
Alta dimensión	Muy mala	Buena
Robustez outliers	Mala	Buena

Cuándo usar cada algoritmo

Usa KNN cuando

- Dataset pequeño ($< 10K$ instancias)
- Necesitas interpretabilidad simple
- Pocas features (< 20)
- Relaciones locales importantes
- Predicción en tiempo real no crítica

Usa SVM cuando

- Dataset mediano (hasta $\sim 100K$)
- Alta dimensionalidad
- Separación clara entre clases
- Necesitas garantías teóricas
- Predicción rápida requerida

Resumen y próximos pasos

Lo que aprendimos

Conceptos fundamentales:

- Clasificación no lineal
- Complejidad del modelo
- Bias-variance tradeoff
- Validación cruzada
- Kernel trick

Habilidades prácticas:

- Implementar KNN y SVM
- Optimizar hiperparámetros
- Diagnosticar sobre/subajuste
- Evaluar correctamente
- Seleccionar algoritmo apropiado

Mensaje clave

No existe el **mejor algoritmo** - todo depende del problema, datos y restricciones computacionales

Preguntas de repaso

- 1 ¿Por qué KNN es considerado *lazy learning*?
- 2 ¿Qué significa que $k = 1$ en KNN maximiza el sobreajuste?
- 3 ¿Cuál es la diferencia entre *hard margin* y *soft margin* SVM?
- 4 ¿Qué son los vectores de soporte?
- 5 ¿Qué hace el *kernel trick* y por qué es importante?
- 6 ¿Cómo afecta γ a un SVM con kernel RBF?
- 7 ¿Por qué es esencial escalar features en SVM?
- 8 ¿Cuándo usarías *LinearSVC* en lugar de *SVC*?
- 9 ¿Qué es validación cruzada estratificada?
- 10 ¿*Random Search* vs *Grid Search*: cuándo usar cada uno?

Literatura recomendada

Libros fundamentales:

- **The Elements of Statistical Learning** - Hastie, Tibshirani, Friedman
 - Capítulo 12: Support Vector Machines
- **Pattern Recognition and Machine Learning** - Bishop
 - Capítulo 7: Sparse Kernel Machines
- **An Introduction to Statistical Learning** - James et al.
 - Capítulo 9: Support Vector Machines

Papers clásicos:

- Cortes & Vapnik (1995): Support-Vector Networks
- Boser et al. (1992): Training Algorithm for Optimal Margin Classifiers

Recursos online:

- Scikit-learn User Guide: SVM
- Andrew Ng - CS229 Lecture Notes

¡Muchas gracias por su atención!

¿Preguntas?



Contacto: Marco Teran
webpage: marcoteran.github.io/
e-mail: mtteranl@eafit.edu.co